

Simulación RTL y dirigida por eventos en C++ para aplicaciones de DSP

Ing. Leonardo Luis Ortiz

Instituto Balseiro
Centro Atómico Bariloche

1 Bit Accurate Data Types

Bit Accurate Data Types

La necesidad de precisión a nivel de bits se hace especialmente evidente ahora que los diseñadores crean hardware directamente desde C++, cuyos tipos nativos solo admiten anchos de 1, 8, 16, 32, etc., bits. Muchos tipos de datos con precisión a nivel de bits que se utilizan actualmente son bibliotecas de clases desarrolladas internamente por las empresas y modelan dicha precisión mediante técnicas tradicionales de desplazamiento y máscara.

Hasta la fecha, existen dos tipos de datos de precisión bit a bit estándar en la industria: **SystemC** y los tipos de datos **Algorithmic C**.

<https://systemc.org/>

<http://hlslibs.org>

Compilation, Debug, and Simulation Speed

Para compilar y usar los tipos de datos de Algorithmic C, el archivo de cabecera correspondiente a los tipos de datos enteros (**ac_int**) o de punto fijo (**ac_fixed**) debe incluirse en el/los archivo(s) fuente de C++.

```
#include <ac_int.h>
#include <ac_fixed.h>
```

También es fundamental para lograr los tiempos de ejecución más rápidos que se establezca el nivel de optimización más alto (-O3 en gcc y /Ox en MS Visual).

Header Files and Typedefs

Aunque los tipos de datos de Algorithmic C se ejecutan mucho más rápido que los tipos de datos de SystemC, en general se ejecutan más lento que los tipos nativos de C++. Por esta razón, además de simplificar la depuración, a menudo es conveniente poder alternar rápidamente entre tipos de datos de Algorithmic C y tipos nativos de C++.

La forma más sencilla de hacerlo es definir todas las variables en un archivo de cabecera global tanto para los tipos de Algorithmic C como para los tipos nativos, y usar definiciones del compilador para alternar entre las dos definiciones. El Ejemplo 1 muestra un archivo de cabecera que usa una definición del compilador, «NATIVE_TYPES», para seleccionar entre las definiciones de tipo de «dType» y «oType», ya sea como tipos nativos de C++ o como tipos de datos de Algorithmic C. Este archivo de cabecera se incluye en el Ejemplo 2, que define todas sus variables en función de las variables definidas con typedef.

Ejemplo 1/2 - Usando Typedef

```
#ifndef __TYPEDFS__
#define __TYPEDFS__
#include <ac_int.h>

#ifdef NATIVE_TYPES
    typedef short int dType;
    typedef int oType;
#else
    typedef ac_int<7,true> dType;
    typedef ac_int<14,true> oType;
#endif

#endif
```

```
#include "typedefs.h"

void test(dType a, dType b, oType &c){
    oType tmp;

    tmp = a*b;
    c = tmp;
}
```

Los tipos de datos enteros de Algorithmic C permiten a los diseñadores modelar un vector de bits con o sin signo con precisión de bits estática. Esto se asemeja mucho a lo que los diseñadores de RTL pueden hacer hoy en día con VHDL y Verilog 2001. Los tipos de datos `ac_int` están basados en templates (plantillas) y permiten a los diseñadores especificar tanto el ancho como el signo de las variables.

Los tipos de datos enteros sin signo de Algorithmic C se declaran como:

```
ac_int<W,false> x;
```

donde:

W = Ancho de bits

$0 \leq x \leq 2^W - 1$ en incrementos de 1

Cualquier valor asignado a “**x**” que sea mayor que el valor máximo representable, o negativo en este caso, provocará un desbordamiento. Este es el mismo comportamiento con el que los diseñadores de RTL están familiarizados hoy en día al crear contadores. El ejemplo 3, que es puramente para simulación, muestra el uso de un entero sin signo de 7 bits para crear una onda sinusoidal.

Ejemplo 3 - Algorithmic C Unsigned Integer

```
#include <ac_int.h>
#include <fstream>
#include <cmath>

using namespace std;

const double pi = 3.14;
const int OFFSET = 64;

int main(){
    fstream fptr;
    fptr.open("tmp.txt", fstream::out);
    ac_int<7,false> x[128];
    for(int i=0;i<128;i++){
        x[i] = OFFSET + 63*sin(2*pi*i/64);
        fptr << x[i] <<endl;
    }
    fptr.close();
}
```

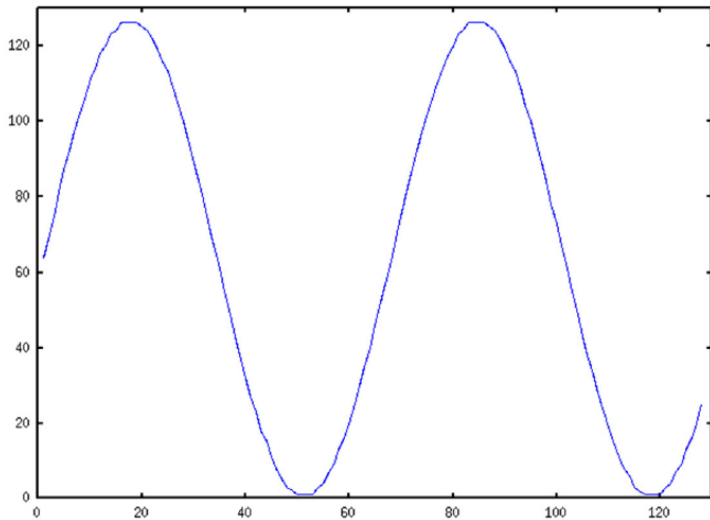


Figura 1: Rango máximo de un entero sin signo de 7-bit

Los efectos del **wrapping** a un tipo de datos con precisión de bits se pueden observar al volver a graficar los resultados del Ejemplo 1 cuando el desplazamiento se incrementa a 80, como se muestra en la Figura 2.

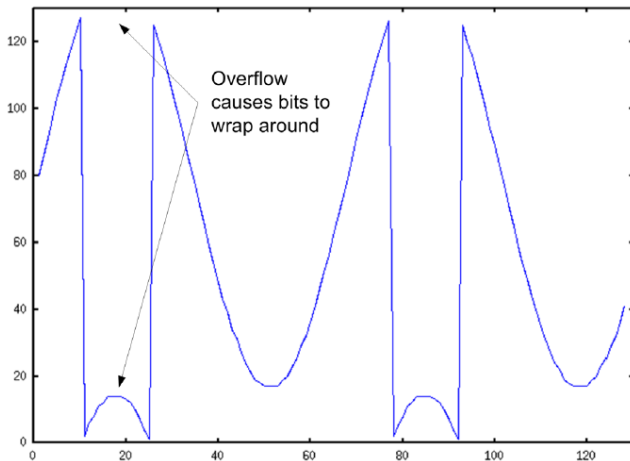


Figura 2: Efecto del wrapping en Tipos de Datos Sin Signo

Entero con signo

Los tipos de datos enteros con signo de Algorithmic C se declaran como:

```
ac_int<W,true> x;
```

donde:

W = Ancho de bits

$-2^{W-1} \leq x \leq 2^{W-1} - 1$ en incrementos de 1

Los tipos de datos enteros con signo y precisión de bits tienen un comportamiento de desbordamiento similar al de los enteros sin signo, con la diferencia de que los tipos de datos con signo se desbordan según la expresión mostrada anteriormente. El ejemplo 4 muestra la generación de una onda sinusoidal donde el tipo de "x" se ha cambiado a entero con signo en la línea 10. El desplazamiento se ha establecido en cero, ya que ahora el tipo de datos admite valores negativos.

Ejemplo 4 - Entero signado en Algorithmic C

```
#include <ac_int.h>
#include <iostream.h>
#include <fstream.h>
#include <math.h>

const double pi = 3.14;
const int OFFSET = 0;

int main(){
    fstream fptr;
    fptr.open("tmp.txt", fstream::out);
    ac_int<7,true> x[128];

    for(int i=0;i<128;i++){
        x[i] = OFFSET + 63*sin(2*pi*i/64);
        fptr << x[i] <<endl;
    }
    fptr.close();
}
```

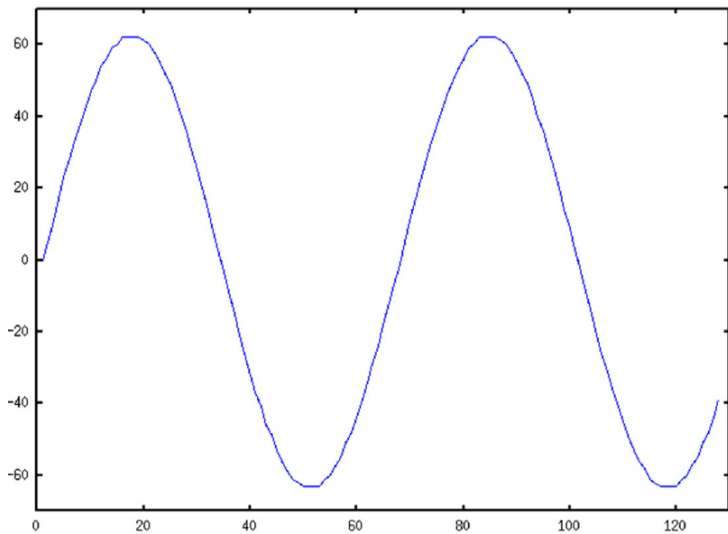


Figura 3: Rango máximo de un entero con signo de 7-bit

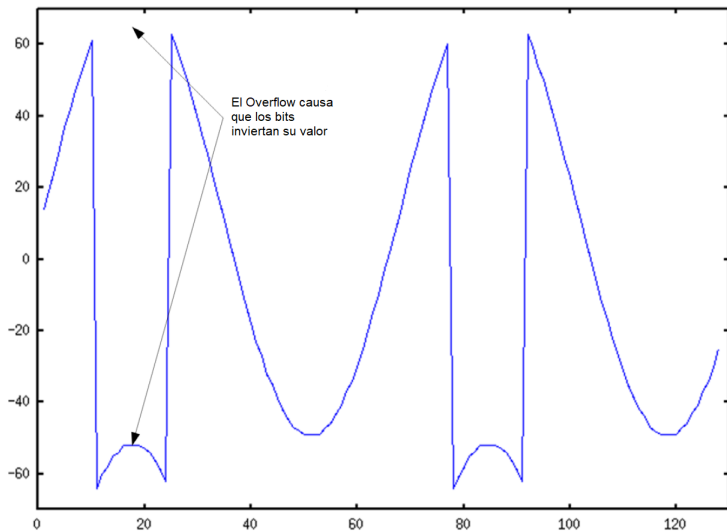


Figura 4: Efectos del wrapping en Tipos de datos con signo de 7-bit

Los tipos de datos de punto fijo de Algorithmic C permiten a los diseñadores modelar un vector de bits con o sin signo con precisión de punto fijo estática. Esto es algo que no se puede hacer directamente en RTL y es una de las muchas ventajas de la síntesis de alto nivel. Aunque la mayoría de los algoritmos DSP se diseñan utilizando aritmética de punto flotante o fijo, la implementación RTL real se realiza con enteros, y el diseñador tiene que controlar manualmente el punto decimal desplazando los resultados intermedios a la izquierda o a la derecha. Este método de diseño no solo es tedioso, sino también propenso a errores. HLS permite a los diseñadores construir hardware directamente a partir de un modelo de punto fijo. Los tipos de datos **ac_fixed** están estandarizados y permiten a los diseñadores especificar tanto el ancho entero como el fraccionario, así como el signo de las variables.

Tipos de datos de Punto Fijo sin signo

Los tipos de datos en punto fijo sin signo de Algorithmic C se declaran como:

```
ac_fixed<W,I,false> x;
```

$0 \leq x \leq (1 - 2^{-W})2^I$ en incrementos de 2^{I-W}

Se especifican tanto el ancho total **W** como el número de bits enteros **I**. Como resultado, **I** determina la posición de la coma decimal con respecto al bit más significativo (MSB).

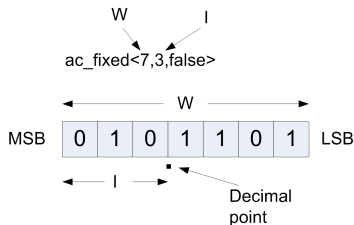


Figura 5: Posición del punto decimal en Punto Fijo

Ejemplo 5 - Entero sin signo en punto fijo en Algorithmic C

```
#include <ac_fixed.h>
#include <iostream.h>
#include <fstream.h>
#include <math.h>

const double pi = 3.14;
const double OFFSET = 1.0;

int main(){
    fstream fptr;
    fptr.open("tmp.txt", fstream::out);
    ac_fixed<7,1,false> x[128];

    for(int i=0;i<128;i++){
        x[i] = OFFSET + 0.98*sin(2*pi*i/64);
        fptr << x[i] <<endl;
    }
    fptr.close();
}
```

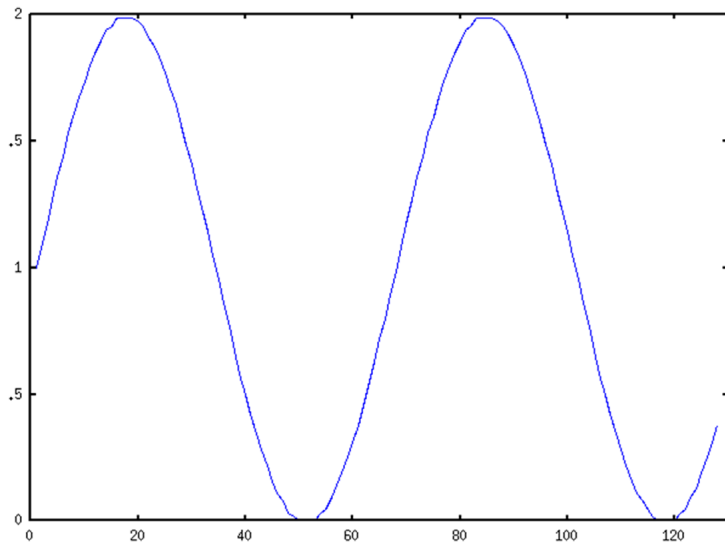


Figura 6: Señal senoidal con punto fijo sin signo de 7-bit

Tipos de datos de Punto Fijo signado

Los tipos de datos en punto fijo sin signo de Algorithmic C se declaran como:

```
ac_fixed<W,I,true> x;
```

$-0,5 * 2^I \leq x \leq (0,5 * 2^{-W})2^I$ en incrementos de 2^{I-W}

I determina la ubicación del punto decimal en relación con el bit más significativo (MSB), que también es el bit de signo.

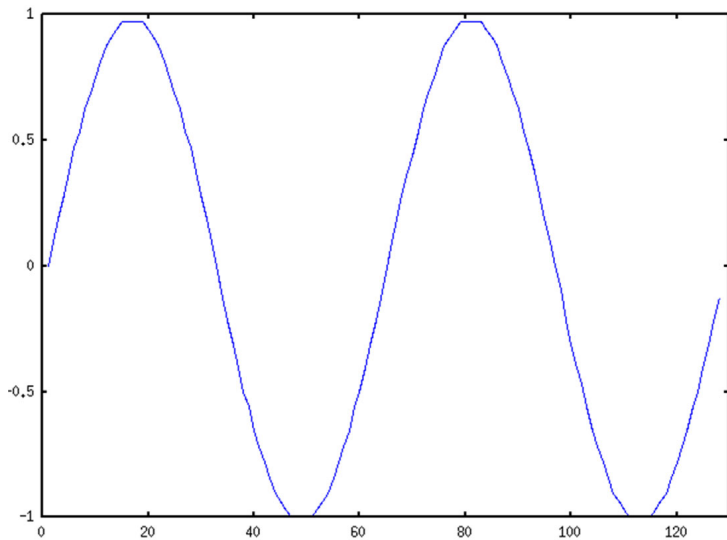


Figura 7: Señal senoidal en punto fijo con signo de 7-bit

Además de permitir expresar algoritmos de forma más natural, los tipos de datos de punto fijo también proporcionan mecanismos para gestionar la cuantización y el desbordamiento. El modo predeterminado de '**ac_fixed**' consiste en truncar y desbordar, de forma similar a lo mostrado para '**ac_int**'. Este modo no consume área adicional, pero puede no ser ideal para algunas aplicaciones. Los tipos de datos **ac_fixed** admiten diversos modos de cuantización y desbordamiento, que se describen en detalle en el manual de tipos de datos de Algorithmic C. En este apartado se presentan las razones por las que podría ser conveniente habilitar estos modos. La cuantización y el desbordamiento se habilitan mediante parámetros de plantilla adicionales para los tipos de datos '**ac_fixed**'.

```
ac_fixed<W,I,S,Q,O> x;
```

donde **Q** y **O** setean los modos de cuantización y overflow, respectivamente.

Truncado y Redondeo

El comportamiento predeterminado es truncar y descartar los bits a la derecha del bit menos significativo (LSB). Esto resulta en una pérdida total de información. Un ejemplo de esto sería asignar datos fraccionarios a una variable de punto fijo con solo bits enteros. Por ejemplo:

```
ac_fixed<7,7,true> x = 0.5;
```

Al imprimir el valor de "x" después de la asignación, se obtiene cero, ya que "x" no contiene decimales. En lugar de descartar los decimales, se puede usar un modo de redondeo para redondear hacia arriba o hacia abajo según su valor. De forma similar a lo que aprendimos en la escuela primaria, podemos redondear hacia arriba o hacia abajo dependiendo de la posición del valor decimal entre dos números enteros.

```
ac_fixed<7,7,true,AC_RND> x = 0.5;
```

AC_RND redondea hacia el infinito positivo, lo que significa que a "x" se le asignará el valor de uno. El modo de redondeo redondea en función del incremento mínimo permitido definido por W e I.

Saturación y Overflow

Los ejemplos anteriores mostraron que el comportamiento predeterminado es que los bits se "desborden" cuando se excede el valor máximo o mínimo representable. Esto se conoce como desbordamiento o subdesbordamiento y suele ser una situación muy indeseable. La mayoría de los algoritmos pueden, y deben, diseñarse para que el desbordamiento nunca ocurra. Esto implica tener en cuenta el rango dinámico de las variables y garantizar que el crecimiento interno de bits sea suficiente para representar todos los rangos posibles de entradas del algoritmo. Sin embargo, existen situaciones en las que es necesario garantizar que el desbordamiento nunca ocurra. Los sistemas de misión crítica, como el control de vuelo, serían un buen ejemplo donde el desbordamiento sería desastroso. Los algoritmos de vídeo son otro ejemplo de por qué el desbordamiento sería indeseable, ya que la mayoría de las personas no desean que los píxeles cambien de los colores más brillantes a los más oscuros.

Se puede evitar el desbordamiento habilitando la saturación en el tipo de datos de punto fijo. Encontrará más detalles sobre el comportamiento de estos modos en el manual de tipos de datos de Algorithmic C. Se debe tener precaución al usar la saturación, ya que suele aumentar el área del diseño. NO habilite la saturación en todas las variables del diseño. La saturación se suele usar selectivamente en algunos lugares. El ejemplo 6 se ha reescrito para habilitar la saturación mediante el parámetro de plantilla de desbordamiento **AC_SAT**, que se muestra en la línea 10 del ejemplo 7.

Tomando el Ejemplo 6 y agregando un pequeño desplazamiento positivo, el resultado se desbordará, lo que dará lugar a un resultado sin sentido o potencialmente catastrófico, como se muestra en la Figura 8.

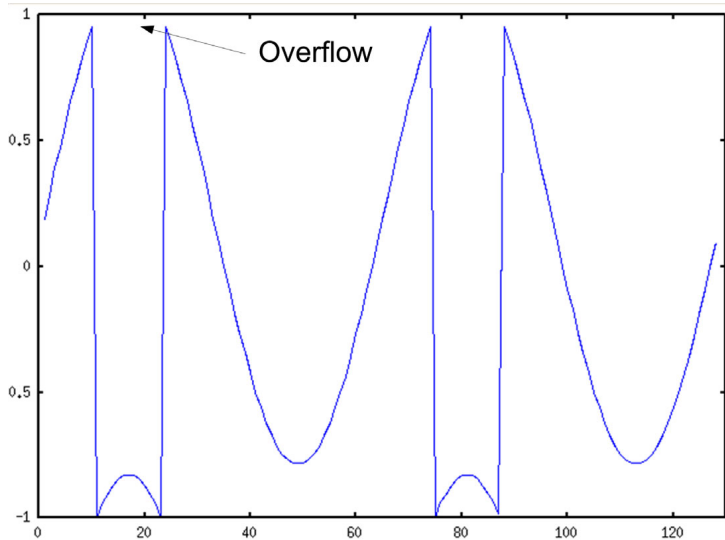


Figura 8: Overflow en punto fijo

Se puede evitar el desbordamiento habilitando la saturación en el tipo de datos de punto fijo. Encontrará más detalles sobre el comportamiento de estos modos en el manual de tipos de datos de Algorithmic C. Se debe tener precaución al usar la saturación, ya que suele aumentar el área del diseño. NO habilite la saturación en todas las variables del diseño. La saturación se suele usar selectivamente en algunos lugares. El ejemplo 6 se ha reescrito para habilitar la saturación mediante el parámetro de plantilla de desbordamiento **AC_SAT**, que se muestra en la línea 10 del ejemplo 7.

Saturación y Overflow

```
#include <ac_fixed.h>
#include <iostream.h>
#include <fstream.h>
#include <math.h>

const double pi = 3.14;
const double OFFSET = 0.2;

int main(){
    fstream fptr;
    fptr.open("tmp.txt", fstream::out);
    ac_fixed<7,1,true,AC_TRN,AC_SAT> x[128];

    for(int i=0;i<128;i++){
        x[i] = OFFSET + 0.98*sin(2*pi*i/(double)64);
        fptr << x[i] <<endl;
    }
    fptr.close();
}
```

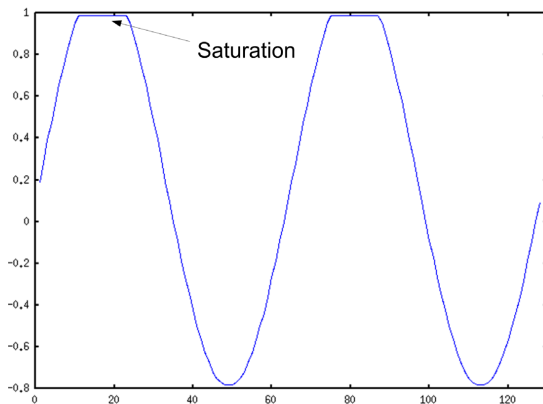


Figura 9: Efecto de Saturación

La saturación solo debe usarse cuando sea absolutamente necesario. La mayoría de los algoritmos pueden diseñarse para evitar el desbordamiento/subdesbordamiento seleccionando los anchos de bits variables adecuados para admitir el rango dinámico completo del algoritmo. La saturación suele afectar tanto al área como al rendimiento.

Todos los operadores aritméticos y lógicos estándar de C++ son compatibles con los tipos de datos `ac_int` y `ac_fixed`. Operadores como la multiplicación y la suma están diseñados para devolver un resultado sin pérdida de precisión. Para una descripción completa de todos los operadores, consulte el manual de referencia de Algorithmic C. Operadores como la división (“/”) y el módulo (“%”) son compatibles, pero deben usarse con precaución cuando los operandos son variables. Esto se debe a que una operación como la división consume mucha memoria, y la mayoría de los diseñadores de hardware no utilizan divisores de memoria. Este es un aspecto de HLS que suele malinterpretarse, ya que es perfectamente razonable escribir algo como `'z = x/y'` en C++. Si realmente se necesita una división, existen implementaciones más eficientes, como el algoritmo CORDIC. La mayoría de las herramientas de HLS proporcionan una biblioteca que implementa estas funciones de forma más eficiente. Las divisiones o el módulo por una constante son mucho más aceptables y se implementan mediante lógica de suma y desplazamiento. La división o el módulo por una potencia de dos se implementan mediante desplazamientos estáticos, sin coste adicional de área. Dos operadores cuyo comportamiento merece mención son la selección de bits y el desplazamiento, pero primero, un breve repaso de los operadores aritméticos.

Operadores aritméticos Bitwise: *, +, -, /, &, |, ^, %

Los operadores aritméticos bit a bit de Algorithmic C están diseñados para que no haya pérdida de precisión en el valor de retorno. Además, se admite la combinación de tipos de datos con y sin signo de Algorithmic C, y se devuelve el signo esperado.

El tipo de retorno de una operación aritmética gestiona automáticamente el crecimiento de bits para evitar la pérdida de precisión. Esto se logra automáticamente porque el ancho de bits de los dos operandos se especifica como parámetros de template al declarar las variables **ac_int**. Esto se muestra a continuación:

```
ac_int<8,true> a,b;    //8-bits signado
```

Multiplicando “a” por “b”, cada uno con precisión de 8 bits, requieres 16 bits de precisión:

```
(returns ac_int<16,true>)(a*b)
```

La suma “a” + “b” requiere 9 bits de precisión:

```
(returns ac_int<9,true>)(a+b)
```

Operador selector de Bit: []

Se pueden leer o escribir bits individuales de un tipo de datos **ac_int** o **ac_fixed** mediante el operador []. El índice del operador selecciona la posición del bit; por ejemplo, 'x[1]', 'x[3]', 'x[7]'. El valor devuelto es un objeto de la clase **ac_int::bitref** y se proporciona una función de conversión integrada a **ac_int** y **bool**. El siguiente fragmento de código muestra cómo se puede usar el operador de selección de bits para leer el bit de signo de un **ac_int**.

```
ac_int<11,true> x;
bool is_neg;

if(x[10]) //test for sign bit treated as bool
    is_neg = true;
else
    is_neg = false;
```

El operador de selección de bit se puede usar con la misma facilidad para escribir un bit en un **ac_int** o **ac_fixed**. Por ejemplo:

```
ac_fixed<9,1,false> x = 0;
bool add_one = true;

if(add_one)
    x[8] = 1;    //set MSB of x
```


Operadores de desplazamiento: «, »

Los operadores de desplazamiento merecen un análisis detallado porque, a diferencia de los operadores aritméticos que mantienen la precisión completa, devuelven la precisión del operando izquierdo. Esto puede generar resultados inesperados. Además, los diseñadores deben prestar atención a cómo utilizan los desplazamientos, ya que ambos operandos pueden ser con o sin signo. Esto también puede dar lugar a resultados sorprendentes que no serían posibles en RTL tradicional.

Desplazamiento a la derecha sin signo: »

El operador de desplazamiento a la derecha sin signo, aplicado a los tipos de datos **ac_int** o **ac_fixed** sin signo, se comporta exactamente como un diseñador RTL esperaría de un desplazador. Cada bit se desplaza a la derecha la cantidad de desplazamiento y se insertan ceros en los bits más significativos.

```
ac_int<4,false> x = -1; //set all bits to 1's
int idx;
ac_int<4,false> y = x >> idx;
```

y	idx	Value of y				
<table><tr><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	1	1	1	1	0	15
1	1	1	1			
<table><tr><td>0</td><td>1</td><td>1</td><td>1</td></tr></table>	0	1	1	1	1	7
0	1	1	1			
<table><tr><td>0</td><td>0</td><td>1</td><td>1</td></tr></table>	0	0	1	1	2	3
0	0	1	1			
<table><tr><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	0	0	0	1	3	1
0	0	0	1			
<table><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	4	0
0	0	0	0			

Figura 10: Desplazamiento a la derecha de números sin signo

Desplazamiento a la derecha con signo: »

El desplazamiento a la derecha con signo tiene un comportamiento algo inesperado. Al desplazar a la derecha un `ac_int` con signo, el bit de signo siempre se conserva, lo que tiene como resultado final el desplazamiento de unos desde el bit más significativo (MSB) en lugar de ceros.

```
ac_int<4,true> x = 0;  
x[3] = 1 //set x equal -8  
int idx;  
ac_int<4,true> y = x >> idx;
```

y	<u>idx</u>	<u>Value of y</u>				
<table><tr><td>1</td><td>0</td><td>0</td><td>0</td></tr></table>	1	0	0	0	0	-8
1	0	0	0			
<table><tr><td>1</td><td>1</td><td>0</td><td>0</td></tr></table>	1	1	0	0	1	-4
1	1	0	0			
<table><tr><td>1</td><td>1</td><td>1</td><td>0</td></tr></table>	1	1	1	0	2	-2
1	1	1	0			
<table><tr><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	1	1	1	1	3	-1
1	1	1	1			
<table><tr><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	1	1	1	1	4	-1
1	1	1	1			

Figura 11: Desplazamiento a la derecha de números con signo

Desplazamiento a la izquierda sin signo: «

Desplazar un entero sin signo `ac_int` a la izquierda y asignar el resultado a una variable con la misma precisión tiene un comportamiento similar al de un desplazamiento sin signo a la derecha, excepto que se insertan ceros desde la posición del bit menos significativo (LSB).

```
ac_int<4,false> x = -1; //set all bits to 1's
int idx;
ac_int<4,false> y = x << idx;
```

y	idx	Value of y				
<table><tr><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	1	1	1	1	0	15
1	1	1	1			
<table><tr><td>1</td><td>1</td><td>1</td><td>0</td></tr></table>	1	1	1	0	1	14
1	1	1	0			
<table><tr><td>1</td><td>1</td><td>0</td><td>0</td></tr></table>	1	1	0	0	2	12
1	1	0	0			
<table><tr><td>1</td><td>0</td><td>0</td><td>0</td></tr></table>	1	0	0	0	3	8
1	0	0	0			
<table><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	4	0
0	0	0	0			

Figura 12: Desplazamiento a la izquierda de números sin signo

Desplazamiento a la izquierda con signo: «

El desplazamiento a la izquierda con signo tiene un comportamiento similar al desplazamiento a la izquierda sin signo, donde se insertan ceros desde el bit menos significativo (LSB).

```
ac_int<4,true> x = -1; //set all bits to 1's
int idx;
ac_int<4,true> y = x << idx;
```

y	idx	Value of y				
<table><tr><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	1	1	1	1	0	-1
1	1	1	1			
<table><tr><td>1</td><td>1</td><td>1</td><td>0</td></tr></table>	1	1	1	0	1	-2
1	1	1	0			
<table><tr><td>1</td><td>1</td><td>0</td><td>0</td></tr></table>	1	1	0	0	2	-4
1	1	0	0			
<table><tr><td>1</td><td>0</td><td>0</td><td>0</td></tr></table>	1	0	0	0	3	-8
1	0	0	0			
<table><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	4	0
0	0	0	0			

Figura 13: Desplazamiento a la izquierda de números con signo

Pérdida inesperada de precisión

El desplazamiento a la izquierda puede tener un comportamiento inesperado, pero correcto, cuando se espera que el resultado se base en la precisión de las variables objetivo. Esto se entiende mejor viendo un ejemplo:

```
ac_int<4,false> x = -1;    //set all bits to 1's
int idx;
ac_int<8,false> y = x << idx;
```

En el ejemplo anterior, “x”, un valor de cuatro bits sin signo, se desplaza a la izquierda y se asigna a “y”, un valor de ocho bits sin signo. Un error común entre los diseñadores que se inician en los tipos de datos de Algorithmic C es creer que los bits más significativos de “x” se almacenan en “y” al desplazarse más allá del bit más significativo (MSB) de “x”. La figura 14 muestra el comportamiento real. Recuerde que el operador de desplazamiento devuelve la precisión del operando izquierdo, que en este ejemplo es de cuatro bits. Para conservar los bits que se desplazan más allá del MSB de “x”, es necesario convertir “x” a la precisión de “y”. Por ejemplo:

```
ac_int<4,false> x = -1;    //set all bits to 1's
int idx;
ac_int<8,false> y = (ac_int<8,false>) x << idx;
```

Pérdida inesperada de precisión

El desplazamiento a la izquierda puede provocar la pérdida de bits si la variable que se está desplazando no tiene la misma precisión que la parte izquierda de la asignación.

Esto ahora da el comportamiento deseado que se muestra en la Figura 15.

y	idx	Value of y								
<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	0	0	0	0	1	1	1	1	0	15
0	0	0	0	1	1	1	1			
<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td></tr></table>	0	0	0	0	1	1	1	0	1	14
0	0	0	0	1	1	1	0			
<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr></table>	0	0	0	0	1	1	0	0	2	12
0	0	0	0	1	1	0	0			
<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	1	0	0	0	3	8
0	0	0	0	1	0	0	0			
<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	0	0	0	0	4	0
0	0	0	0	0	0	0	0			

Figura 14: Pérdida inesperada de precisión cuando se desplaza a la izquierda

Pérdida inesperada de precisión

y	idx	Value of y								
<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	0	0	0	0	1	1	1	1	0	15
0	0	0	0	1	1	1	1			
<table><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td></tr></table>	0	0	0	1	1	1	1	0	1	30
0	0	0	1	1	1	1	0			
<table><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr></table>	0	0	1	1	1	1	0	0	2	60
0	0	1	1	1	1	0	0			
<table><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td></tr></table>	0	1	1	1	1	0	0	0	3	120
0	1	1	1	1	0	0	0			
<table><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	1	1	1	1	0	0	0	0	4	240
1	1	1	1	0	0	0	0			

Figura 15: Casteado a la precisión deseada cuando se desplaza a la izquierda

Los tipos de datos de Algorithmic C proporcionan varios métodos integrados que se describen en detalle en el manual de referencia.

<https://hlslibs.org/>

Los métodos más utilizados para el diseño y la simulación se describen a continuación.

El método de lectura de segmento tiene la forma: *slc* < *W* > (*intlsb*)

Donde el parámetro de plantilla “W” especifica el ancho del segmento, y “lsb” indica dónde comienza el segmento. No es posible ajustar dinámicamente el ancho del segmento porque “W” es un parámetro de plantilla. Sin embargo, sí es posible cambiar dinámicamente el valor de “lsb”. Considere el siguiente ejemplo donde se lee un segmento de tres bits de “x” y se asigna a “y”. “lsb” apunta a la posición del bit dos. La figura muestra gráficamente cómo funciona la lectura del segmento.

```
ac_int<8,false> x = 100;  
ac_int<3,false> y;  
  
y = x.slc<3>(2);
```

Slice Read: slc

El método de lectura de segmento tiene la forma: $slc < W > (intlsb)$

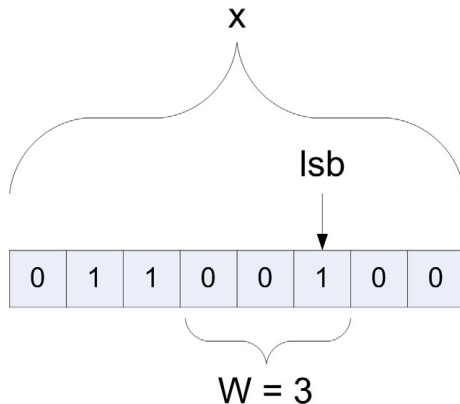


Figura 16: Lectura de segmento

Problemas con la compilación del método Read Slice

Algunos compiladores pueden generar errores cuando se utiliza el método **read slice** dentro de una función o clase con plantillas. Si esto ocurre, simplemente coloque la palabra clave **template** antes del nombre del método.

```
ac_int<8,false> x = 100;  
ac_int<3,false> y;  
  
y = x.template slc<3>(2);
```

Slice Write: set_slc

El método `set_slc` tiene la forma: `set_slc(int lsb, const ac_int<W,S> &slc)`

Donde **lsb** es el índice del vector de bits e indica dónde se debe escribir el segmento. Este comportamiento es esencialmente el mismo que el mostrado para las lecturas de segmentos en la Figura 16. El segmento que se escribe puede ser un entero con o sin signo (**ac_int**). Al igual que con las lecturas de segmentos, el tamaño del segmento no se puede modificar dinámicamente. Su uso es el siguiente:

```
ac_int<8,false> x = 0;
ac_int<3,false> y = 5;

x.set_slc(2,y);
```

Funciones de conversión explícitas

Los tipos de datos de Algorithmic C proporcionan varias funciones de conversión implícitas y explícitas. Estas se encuentran completamente documentadas en el manual de referencia del lenguaje. Las funciones de conversión explícitas suelen ser necesarias al asignar un **ac_fixed** a un **ac_int**, o al intentar usar **printf** para imprimir los resultados de una simulación. El primer caso genera un error de compilación, y el segundo, al usar **printf**, provoca un error de segmentación en tiempo de ejecución.

```
ac_fixed<8,3,false> x = 3.185;  
ac_int<8,false> y;
```

El asignamiento que aparece a continuación genera un error de compilación.

```
y = x;
```

Debería escribirse

```
y = x.to_int();
```

La siguiente instrucción provoca un error de segmentación en tiempo de ejecución:

```
printf(" %d\n", y);
```

Debería escribirse

```
printf(" %d\n", y.to_int());
```

Funciones de conversión implícitas

Los tipos de datos de algorithmic C tienen funciones de conversión integradas para el operador de asignación “=” hasta 64 bits. Estas funciones convierten automáticamente los tipos de datos algorithmic C de vuelta a tipos nativos. Esto permite usar variables directamente para comprobar condicionalmente cualquier bit activado en el vector de bits. Por ejemplo:

```
ac_int<64,false> ctrl = 37;  
if(ctrl) //test for any bit set to a one  
    tmp += din;
```

Un error de compilación ocurre si “ctrl” es mayor que 64 bits. En este caso, debe utilizarse una de las funciones de conversión explícitas.

Las librerías Algorithmic C proveen algunas funciones útiles para el diseño de hardware.

Array Uninitialization: `ac::init_array`

Esta función permite inicializar y desinicializar un array con una constante o un valor irrelevante. La ventaja de usar esta función integrada es que la herramienta HLS puede optimizar el array de forma más eficiente, ya que no necesita analizar bucles ni asignaciones para determinar la intención. El uso más común de esta función es desinicializarlo, es decir, asignar valores irrelevantes a todos sus elementos. Esto se hace para evitar que el hardware borre todas las ubicaciones de un array estático mapeado en memoria. Por definición en el lenguaje C++, una variable declarada como estática se inicializa a cero al ser creada. En muchos casos, no es necesario liberar la memoria de un diseño, ya que se sabe que la memoria se escribe antes de leerse. De este modo, se puede eliminar la sobrecarga de recorrer inicialmente cada ubicación de memoria. La función `ac::init_array` tiene la siguiente forma:

```
bool ac::init_array<"init value">("base address of array", "
    number of elements");
```

Considere el siguiente fragmento de código donde la matriz “a” se asigna a una memoria.

```
static int a[1000];
static bool dummy = ac::init_array<AC_VAL_DC>(a,1000);
```

A menudo es necesario calcular estáticamente el número mínimo de bits requeridos para un tipo de datos de Algorithmic C con el fin de indexar una matriz. Se proporcionan las siguientes funciones:

- `ac::log2_ceil<x>::val` - returns the number of bits required to index “x” elements.
- `ac::log2_floor<x>::val` - returns $\log_2(x)$.
- `ac::nbits<x>::val` - returns number of bits required to represent “x”.

En todos estos casos, “x” debe ser estáticamente determinable. Por ejemplo:

```
const int WORDS = 175;
void foo(int data[WORDS],
ac_int<ac::log2_ceil<WORDS>::val,false> idx,
int &dout){
    dout = data[idx];
}
```

En el ejemplo anterior, “idx” se ha definido con el número mínimo de bits necesarios para indexar los elementos “WORDS”.

Las bibliotecas de tipos de datos de precisión de bits de Algorithmic C también admiten tipos de datos complejos, lo que elimina la necesidad de que los diseñadores desarrollen sus propias clases complejas. El tipo de datos **ac_complex** es una clase con plantillas que se puede usar con **ac_int** y **ac_fixed**, así como con tipos nativos de C++. Para obtener información sobre su uso y restricciones, consulte el manual de tipos de datos de Algorithmic C.

- High-Level Synthesis Blue Book - Michael Fingeroff (2010) Ed. Xlibris Us
- <https://hlslibs.org/>
- http://www.mentor.com/products/esl/high_level_synthesis/ac_datatypes

Q&A